Milestone 1 — Literature Review and Sequential Baseline

Project: Parallel Multi-Asset Options Pricing Engine (Black–Scholes PDE Stencil Computation)

Course: `CS F422` · **Group:** `group no.` **Members:** `Megh Shah,2024A7PS0476G`
Date: `07/09/2026`

Repository: `https://github.com/Megh-1/Pc_Project`

---

1. Problem statement

Exotic derivatives — American options, basket options on several correlated underlyings, barrier options — have no closed-form price. The standard production approach is to discretise the Black–Scholes partial differential equation on a grid of underlying prices and march backwards in time from the payoff at maturity.

Under a change of variables the Black–Scholes PDE is a convection–diffusion equation, structurally identical to the heat equation. Discretising it with finite differences therefore produces exactly the class of computation this course targets: a **regular-grid stencil sweep**, repeated for thousands of time steps, with a strict sequential dependence between steps and full data parallelism within each step.

The parallel-computing content of the project is that structure. The financial content supplies the boundary conditions, the accuracy requirements, and the reason anyone cares about the latency.

Scope by dimension:

Assets	PDE	Stencil	Milestone
1	1D convection–diffusion	3-point	1 (baseline), 2 (OpenMP/CUDA)
2	+ cross-derivative from correlation	9-point	1 (baseline), 3
3	+ 3 cross-derivatives	27-point	3 (MPI halo exchange)

---

2. Literature review

2.1 The pricing equation

Black and Scholes [1] and Merton [2] showed that a European derivative $V(S,t)$ on an underlying following geometric Brownian motion satisfies

$$\frac{\partial V}{\partial t} + \tfrac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + (r-q)S\frac{\partial V}{\partial S} - rV = 0,$$

with $V(S,T)$ given by the payoff. This is a *terminal*-value problem, solved backwards from $t = T$.

2.2 Reduction to a heat-equation stencil

Two substitutions turn this into a constant-coefficient problem.

Step 1 — reverse time. Let $\tau = T - t$:

$$\frac{\partial V}{\partial \tau} = \tfrac{1}{2}\sigma^2 S^2 V_{SS} + (r-q)SV_S - rV.$$

The equation is now a forward diffusion problem — well-posed for explicit marching, ill-posed backwards, which is why the time loop is irreducibly sequential.

Step 2 — log-price coordinates. Let $x = \ln S$. Then $S\partial_S = \partial_x$ and $S^2\partial_{SS} = \partial_{xx} - \partial_x$, giving

$$\boxed{\frac{\partial V}{\partial \tau} = \tfrac{1}{2}\sigma^2 V_{xx} + \left(r - q - \tfrac{1}{2}\sigma^2\right) V_x - rV}$$

Every coefficient is now **constant in space**. This matters for performance, not just for elegance: the stencil weights (a, b, c) are computed once and reused at every node, so the kernel loads no per-node coefficient array. The result is a pure streaming stencil with very low arithmetic intensity — the canonical memory-bound kernel described by Datta et al. [3].

(An additional exponential substitution $V = e^{\alpha x + \beta\tau}u$ removes the convection and reaction terms entirely, reducing the problem to $u_\tau = \frac{1}{2}\sigma^2 u_{xx}$. We retain the convection form because it generalises directly to the multi-asset case, where the transformation is messier.)

2.3 Discretisation schemes

Scheme	Time order	Space order	Stability	Parallel structure
Explicit Euler	1	2	Conditional: $\Delta\tau \lesssim \Delta x^2/\sigma^2$	Embarrassingly parallel per step
Implicit Euler	1	2	Unconditional	Tridiagonal solve (sequential sweep)
Crank–Nicolson [4]	2	2	Unconditional (A-stable)	Tridiagonal solve

Design decision. We implement both, and designate **explicit Euler as the parallelisation target**, with Crank–Nicolson retained as an accuracy reference. The reasoning:

- The explicit update at each node depends only on the previous time level. Every node in a sweep is independent — a perfect fit for OpenMP threads and CUDA blocks.
- The implicit schemes require a tridiagonal solve. The Thomas algorithm is $O(n)$ but carries a loop-carried dependency in its forward sweep, so it does not parallelise directly. Parallel cyclic reduction (PCR) exists but costs $O(n \log n)$ work and complicates the memory-tiling study that is the point of Milestone 3.
- The stability restriction $\Delta\tau \propto \Delta x^2$ is normally seen as a drawback. Here it is an asset: refining the grid $2\times$ requires $4\times$ more time steps, so total work scales as $O(N^3)$ in 1D and $O(N^4)$ in 2D. This produces genuinely large problems at modest grid sizes, which is what the benchmarking in Milestone 4 needs.

Standard treatments of these schemes in a finance setting are Wilmott, Howison and Dewynne [5], Tavella and Randall [6], and Duffy [7].

2.4 Boundary conditions — the "ghost cells" of the brief

The financial domain $S \in (0, \infty)$ is truncated to $x \in [\ln S_0 - L,\ \ln S_0 + L]$ with $L = 5\sigma\sqrt{T}$, i.e. five standard deviations of terminal log-return. Dirichlet conditions are imposed at the two boundary nodes; these are the ghost cells, and they encode extreme market states:

Boundary	Market state	European call	European put
$S \to 0$	Underlying collapses	$V = 0$	$V = Ke^{-r\tau} - Se^{-q\tau}$
$S \to \infty$	Deep in the money	$V = Se^{-q\tau} - Ke^{-r\tau}$	$V = 0$

For the two-asset exchange option $\max(S_1 - S_2, 0)$ the four edges are $V = 0$ at $S_1 \to 0$ and $S_2 \to \infty$, $V = S_1 e^{-q_1 \tau}$ at $S_2 \to 0$, and $V = S_1 e^{-q_1 \tau} - S_2 e^{-q_2 \tau}$ at $S_1 \to \infty$.

Implication for Milestone 3. In the MPI decomposition these boundary values are analytic and require no communication, but the *interior* subdomain interfaces do — each rank needs a one-cell halo per neighbour per time step (width 1 for the 3-point stencil, and the diagonal weights of the 9- and 27-point stencils make the halo a full corner-inclusive exchange).

2.5 American exercise

An American option satisfies a linear complementarity problem rather than a PDE. With an explicit scheme the standard treatment is a projection step after each sweep, $V_i \leftarrow \max(V_i, \text{payoff}(S_i))$, following Brennan and Schwartz [8]. This is pointwise and adds one comparison per node — it preserves the parallel structure exactly, which is a further argument for the explicit scheme.

2.6 Multi-asset extension

For d correlated assets with correlation matrix ρ, in log coordinates $x_k = \ln S_k$:

$$\frac{\partial V}{\partial \tau} = \tfrac{1}{2} \sum_k \sigma_k^2 V_{x_k x_k} + \sum_{k<l} \rho_{kl} \sigma_k \sigma_l V_{x_k x_l} + \sum_k \left(r - q_k - \tfrac{\sigma_k^2}{2} \right) V_{x_k} - rV$$

The cross-derivative terms are what expand the stencil: they are discretised with the four-corner formula

$$V_{xy}\big|_{i,j} \approx \frac{V_{i+1,j+1} - V_{i+1,j-1} - V_{i-1,j+1} + V_{i-1,j-1}}{4\,\Delta x\,\Delta y}$$

turning a 5-point stencil into a 9-point one in 2D, and a 7-point into a 27-point one in 3D. Craig and Sneyd [9] give the ADI treatment for the implicit case; we use the explicit form.

2.7 Stencil computation on parallel hardware

Micikevicius [10] establishes the reference pattern for GPU finite-difference stencils: shared-memory tiling with halo loading, and register-based streaming along the slowest-varying dimension. Datta et al. [3] give the auto-tuning study across multicore and GPU architectures. Williams, Waterman and Patterson [11] supply the Roofline model we use in §6 to argue that this kernel is bandwidth-limited and therefore that the Milestone 3 optimisations should target memory traffic, not FLOP count.

---

3. Numerical formulation as implemented

3.1 Grid

$N+1$ nodes, $x_i = x_{\min} + i\Delta x$, $S_i = e^{x_i}$, with N even so that the spot price S_0 lands **exactly** on node $N/2$. This removes interpolation error from the accuracy metric — the reported error is purely discretisation error.

Time: M steps of size $\Delta\tau = T/M$, marching $\tau : 0 \to T$.

3.2 Explicit stencil

$$V_i^{n+1} = a\, V_{i-1}^n + b\, V_i^n + c\, V_{i+1}^n$$

$$a = \Delta\tau\left(\frac{\sigma^2}{2\Delta x^2} - \frac{r - q - \sigma^2/2}{2\Delta x}\right), \quad b = 1 - \Delta\tau\left(\frac{\sigma^2}{\Delta x^2} + r\right), \quad c = \Delta\tau\left(\frac{\sigma^2}{2\Delta x^2} + \frac{r - q - \sigma^2/2}{2\Delta x}\right)$$

Stability. Positivity of the scheme (a sufficient condition, and the one we enforce) requires $b \geq 0$, i.e.

$$\Delta\tau \leq \left(\frac{\sigma^2}{\Delta x^2} + r\right)^{-1}$$

The solver computes M from this bound automatically with a 5% safety margin.

3.3 Greeks

Delta and Gamma are recovered from the converged grid by finite differencing in log space:

$$\Delta = \frac{1}{S}\frac{\partial V}{\partial x}, \qquad \Gamma = \frac{1}{S^2}\left(\frac{\partial^2 V}{\partial x^2} - \frac{\partial V}{\partial x}\right)$$

This is free — one extra stencil evaluation at a single node — and is the feature that distinguishes a PDE engine from a Monte Carlo engine for the risk use case described in §8.

---

4. Metrics

4.1 Accuracy

Ground truth is the closed-form price where one exists:

- **European vanilla:** Black–Scholes–Merton formula.

- **Two-asset exchange option** $\max(S_1 - S_2, 0)$: Margrabe's formula [12]. This is the only two-asset payoff with a closed form, which makes it the correct validation target for the correlated 2D solver — a true basket option has none.
- **American put:** no closed form. Validated qualitatively (early-exercise premium must be strictly positive) and against published binomial-tree benchmark values.

Reported quantities: absolute error in price, absolute error in Delta and Gamma, and the empirical order of convergence

$$p = \frac{\log(e_1/e_2)}{\log(\Delta x_1/\Delta x_2)}$$

which must approach 2 for second-order central differences.

4.2 Performance

- **Wall-clock latency**, `clock_gettime(CLOCK_MONOTONIC)`, solver loop only (grid setup and allocation excluded).
- **MLUP/s** — million lattice-site updates per second. The architecture-neutral throughput measure for stencil codes; this is the primary metric carried through all four milestones.
- **GFLOP/s** — 5 flops per update (3 multiplies, 2 adds) in 1D; 17 in 2D.
- **Effective bandwidth** — 16 B/update in 1D (8 read + 8 write per node; the two neighbours are cache-resident in a streaming sweep).
- **Arithmetic intensity** — $5/16 = 0.31$ flop/byte in 1D.

---

5. Sequential baseline

`bs_fdm.c`, ~600 lines of C11, no dependencies beyond libm. Built with `gcc -O3 -march=native -std=c11 bs_fdm.c -o bs_fdm -lm`.

Implemented:

- Explicit Euler, 1D, log coordinates, 3-point stencil
- Crank–Nicolson, 1D, with Thomas tridiagonal solver
- American exercise via post-step projection
- Two correlated assets, explicit, 9-point stencil with correlation term
- Black–Scholes and Margrabe closed forms for validation
- Convergence, benchmark and validation harnesses (`price`, `american`, `converge`, `bench`, `2d` modes)

The three hot loops that become the parallel kernels in Milestone 2 are isolated into standalone functions (`step_explicit_1d`, the American projection loop,

and `step_explicit_2d`) and marked `PARALLEL HOTSPOT` in the source. No other code needs to change to add OpenMP.

Test case throughout: $S_0 = K = 100$, $r = 0.05$, $q = 0$, $\sigma = 0.20$, $T = 1$ year.

Machine: Intel(R) Core(TM) Ultra 5 125H (14 cores [4P + 8E + 2LPE], 18 threads, base 1.20 GHz, boost up to 4.50 GHz; Caches: L1d 432 KiB, L1i 576 KiB, L2 18 MiB, L3 18 MiB; RAM: 16 GB; Compiler: GCC 13.3.0 (`gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0`), flags: `-O3 -march=native -std=c11 -Wall -Wextra -lm` on Ubuntu 24.04 LTS (WSL2, Linux 6.6.87 x86_64))

---

6. Results

6.1 Accuracy — European call, $N = 1024$

Quantity	Analytical	Explicit Euler	Crank–Nicolson
Price	10.45058357	10.45059265	10.45049783
Absolute error	—	9.07 × 10	8.57 × 10
Delta	0.63683065	0.63683300	0.63683133
Gamma	0.01876202	0.01876155	0.01876224

Both schemes reproduce the analytical price to better than 10 absolute — around 0.001% of the option value, comfortably inside a market bid–ask spread. Greeks agree to five decimal places without any additional solves.

Crank–Nicolson shows a *larger* error here only because it was run with $M = N$ time steps while the explicit scheme was forced to $M = 11{,}038$ by its stability bound. Per unit of work, CN is far more accurate; the explicit scheme buys its accuracy with brute force. This is the trade-off the parallelisation is meant to make irrelevant.

6.2 Convergence

N	M	Δx	Absolute error	Observed order
128	173	1.562 × 10 ^{2}	5.710 × 10	—
256	690	7.812 × 10 ^{3}	1.455 × 10	1.97
512	2 760	3.906 × 10 ^{3}	3.626 × 10	2.00
1 024	11 038	1.953 × 10 ^{3}	9.075 × 10	2.00
2 048	44 151	9.766 × 10	2.269 × 10	2.00

Observed order converges to exactly 2.00, confirming the discretisation is correctly second-order in space. Note the M column: it quadruples with each grid refinement, exactly as the stability bound predicts.

(Figure 1 — `fig1_convergence.png`)

6.3 American exercise

	Price
European put, analytical	5.57352602
European put, FDM	5.57352178 (error 4.2×10)
American put, FDM	6.09041128
Early-exercise premium	**0.51688950**

The premium is strictly positive as required, and 6.0904 agrees with published binomial-tree benchmarks for these parameters to four decimal places.

6.4 Baseline throughput — 1D, single thread

N	M	Updates	Time (s)	MLUP/s	GFLOP/s	GB/s
256	690	1.76×10	0.0001	1 905	9.5	30.5
512	2 760	1.41×10	0.0008	1 745	8.7	27.9
1 024	11 038	1.13×10	0.0057	1 992	10.0	31.9
2 048	44 151	9.04×10	0.0294	3 074	15.4	49.2

(Figure 2 — `fig2_baseline_scaling.png`)

Two observations that shape Milestones 2–4, and which we flag explicitly because they complicate the naive plan:

1. **The 1D case is cache-resident, not bandwidth-bound.** At $N = 2048$ the working set is two arrays of 16 KB — it fits in L1/L2 entirely. The 28–49 GB/s figures represent *cache* bandwidth, not DRAM bandwidth. The Roofline argument for memory-bound behaviour only bites once the working set exceeds last-level cache, which in 1D would require $N \sim 10^6$.

2. **`-O3 -march=native` already auto-vectorised the kernel.** The constant stencil weights and `restrict`-qualified pointers let GCC emit AVX FMA instructions unaided; ~1.7–3.1 GLUP/s at single-thread scalar rates would be impossible otherwise. The "apply AVX vectorisation" task in Milestone 3 must therefore be reframed as *verifying and guaranteeing* vectorisation (via intrinsics or `#pragma omp simd`, and confirming with `-fopt-info-vec` and VTune) rather than claiming a speedup that the compiler already delivered. **We will report the auto-vectorised build as the honest baseline**, not a `-O0` or scalar build, since inflating the baseline would invalidate every speedup number in Milestone 4.

Consequence: the multi-asset (2D/3D) case is the real target for parallelisation, and the benchmark ladder in Milestone 4 should be weighted toward $256^2 \to 512^3$ as the brief specifies, with the 1D case used only to validate correctness and thread-scheduling overhead.

6.5 Two correlated assets — validation and runtime

$S_1 = 100$, $S_2 = 95$, $\sigma_1 = 0.25$, $\sigma_2 = 0.30$, $\rho = 0.40$, $T = 1$. Margrabe closed form: **14.45045433**.

N	M	Updates	FDM price	Abs. error	Time (s)	MLUP/s
64	171	6.79×10	14.46789242	1.74×10^{-2}	0.0007	969
128	683	1.10×10	14.45607706	5.62×10^{-3}	0.0093	1 178
256	2 731	1.78×10	14.45304194	2.59×10^{-3}	0.1462	1 215
512	10 923	2.85×10	14.45230032	1.85×10^{-3}	2.8700	994

(Figure 3 — `fig3_2d_runtime.png`)

The 9-point stencil runs at roughly a third the update rate of the 3-point stencil, consistent with its higher operand count and the strided access pattern of the diagonal terms.

Error decreases but stalls around 2×10^{-3} rather than continuing at $O(\Delta x^2)$. This is **domain-truncation error**, not a discretisation bug: at $L = 4$ standard deviations the artificial Dirichlet boundary contributes a fixed error floor that refinement cannot remove. Milestone 2 will re-run this study with $L = 6$ to confirm the floor moves, which also separates truncation error from discretisation error cleanly in the accuracy report.

Runtime grows as $O(N^4)$ — a 512^2 grid already costs 2.87 s for a single option. A market-making desk repricing a book of several thousand positions needs the full grid in well under a second, and a 3-asset 512^3 grid is roughly 5×10^5 times the work of 512^2. **This is the quantitative justification for the entire parallelisation effort.**

---

7. Bottleneck analysis and plan for Milestone 2

Property	Value	Consequence
Arithmetic intensity (1D)	0.31 flop/byte	Bandwidth-bound at scale
Arithmetic intensity (2D)	~1.06 flop/byte	Still well left of ridge point
Inter-step dependency	Strict	Time loop cannot be parallelised
Intra-step dependency	None	Full data parallelism per sweep
Halo width	1 cell (corner-inclusive in 2D/3D)	Cheap MPI exchange, good compute/comm ratio

Milestone 2 plan:

1. `#pragma omp parallel for simd schedule(static)` on `step_explicit_1d` and the outer j loop of `step_explicit_2d`. Hoist the parallel region outside the time loop with a barrier per step, to measure fork–join overhead separately from the sweep itself.
2. Strong-scaling study, $1 \to$ all cores, at fixed N; weak-scaling at fixed work per thread.
3. Naive CUDA kernel: one thread per grid node, global-memory reads only, no tiling. This is the deliberate strawman that Milestone 3's shared-memory tiling is measured against.
4. Measure the thread-scheduling overhead the brief asks for by timing a no-op sweep at the same grid size.

Open risk to flag now: with 44 000 time steps at $N = 2048$, kernel-launch latency (~5–10 µs) will dominate the naive CUDA version at small grids. We expect the GPU to *lose* to the CPU below roughly $N = 10^4$ in 1D. That is a result worth reporting rather than an embarrassment, and it further motivates moving the headline benchmarks to 2D/3D.

---

8. Application context: Risk-as-a-Service for crypto derivatives

Crypto options markets trade continuously, at implied volatilities several times those of equity index options, and increasingly in structured and multi-asset forms where closed-form pricing fails. Smaller prop desks and DeFi market makers need the Greeks of a full portfolio recomputed as the market moves, but cannot justify an in-house compute cluster.

A PDE engine is the right tool for this specifically because §3.3 holds: the Greeks come out of the same grid solve as the price, at essentially zero extra

cost, whereas a Monte Carlo engine must re-simulate per sensitivity. A GPU-accelerated PDE solver exposed behind a low-latency gRPC API — parameters in, price and Greeks out — is a plausible product built directly on this codebase.

Commercial viability is outside the scope of this report; the relevant point for Milestone 4 is that the **latency target is set by the application**: sub-100 ms for a full portfolio revaluation is the number the benchmarks should be compared against.

---

9. Repository

```
.
  bs_fdm.c              # sequential baseline solver
  make_figures.py       # generates report figures from measured output
  Makefile
  results/              # raw timing output
  docs/milestone1.pdf   # this report
```

Reproduce all tables in §6:

```
make
./bs_fdm price     > results/price.txt
./bs_fdm converge  > results/converge.txt
./bs_fdm american  > results/american.txt
./bs_fdm bench     > results/bench.txt
./bs_fdm 2d        > results/2d.txt
python3 make_figures.py
```

---

10. References

1. F. Black and M. Scholes, "The Pricing of Options and Corporate Liabilities," *Journal of Political Economy*, 81(3), 1973.
2. R. C. Merton, "Theory of Rational Option Pricing," *Bell Journal of Economics and Management Science*, 4(1), 1973.
3. K. Datta, S. Kamil, S. Williams, L. Oliker, J. Shalf and K. Yelick, "Optimization and Performance Modeling of Stencil Computations on Modern Microprocessors," *SIAM Review*, 51(1), 2009.
4. J. Crank and P. Nicolson, "A Practical Method for Numerical Evaluation of Solutions of Partial Differential Equations of the Heat-Conduction Type," *Proc. Cambridge Philosophical Society*, 43(1), 1947.
5. P. Wilmott, S. Howison and J. Dewynne, *The Mathematics of Financial Derivatives: A Student Introduction*, Cambridge University Press, 1995.
6. D. Tavella and C. Randall, *Pricing Financial Instruments: The Finite Difference Method*, Wiley, 2000.

7. D. J. Duffy, *Finite Difference Methods in Financial Engineering: A Partial Differential Equation Approach*, Wiley, 2006.
8. M. J. Brennan and E. S. Schwartz, "The Valuation of American Put Options," *Journal of Finance*, 32(2), 1977.
9. I. J. D. Craig and A. D. Sneyd, "An Alternating-Direction Implicit Scheme for Parabolic Equations with Mixed Derivatives," *Computers & Mathematics with Applications*, 16(4), 1988.
10. P. Micikevicius, "3D Finite Difference Computation on GPUs using CUDA," *Proc. 2nd Workshop on General Purpose Processing on Graphics Processing Units (GPGPU-2)*, 2009.
11. S. Williams, A. Waterman and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, 52(4), 2009.
12. W. Margrabe, "The Value of an Option to Exchange One Asset for Another," *Journal of Finance*, 33(1), 1978.